Resilience is one of the goals for moving to the cloud and one of the properties characterizing cloud native applications. Our systems should be resilient to failures and stable enough to ensure a certain service level to their users. The integration points between services over a network are among the most critical areas for achieving a stable and resilient system for production. It's so important that Michael T. Nygard spends a large part of his book *Release It! Design and Deploy Production-Ready Software* (Pragmatic Bookshelf, 2018) on the subject.

This chapter will focus on building resilient, scalable, and efficient applications for the cloud using the reactive paradigm. First I'll introduce the event loop model and the main features of Reactive Streams, Project Reactor, and the Spring reactive stack. Then you'll build a reactive Order Service application using Spring WebFlux and Spring Data R2DBC.

Order Service will interact with Catalog Service to check the availability of books and their details, so you'll see how to implement a reactive REST client using Spring WebClient. The integration point between the two services is a critical area that needs extra care to achieve robustness and fault tolerance. Relying on the Reactor project, you'll adopt stability patterns like retries, timeouts, and failovers. Finally, you'll write autotests to verify the behavior of a reactive application using Spring Boot and Testcontainers.

NOTE The source code for the examples in this chapter is available in the Chapter08/08-begin and Chapter08/08-end folders, containing the initial and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).

8.1 Asynchronous and non-blocking architectures with Reactor and Spring